

Mini-CDC · WBS · Estimation · Planifcation

Gantt

Nom du projet : Task Reminder

Membre	Rôle
Badiaa El Amraoui	chef du projet
Douaa El Idrissi	Product Owner
Sidi Mohamed Moustapha	Développeur/testeur
Mohamed Saad Fakhdi	Développeur/testeur

I. Livrable 1 Mini-Cahier des Charges (Mini-CDC)

1. Contexte du projet :

De nombreuses personnes ont besoin de gérer leurs tâches, mais elles oublient souvent de les mémoriser et de les effectuer malgré l’utilisation des applications ou des sites web classiques de gestion des tâches. C’est pourquoi nous avons pensé à créer une application web qui leur rappellera de réaliser leurs tâches.

2. Parties prenantes :

Rôle	Besoin	Implication
Utilisateur final	Créer, modifier, supprimer des tâches facilement. Recevoir des rappels par notification web (toast, alert, ou email).	Teste l’application sur navigateur (Chrome, Firefox, Edge), donne son avis.
Chef de projet	Suivre l'avancement et la qualité, respecter les délais (S12), gérer les risques . S'assurer que les livrables sont complets.	Planifie les sprints, anime les réunions d'équipe, vérifie la cohérence entre WBS, Gantt et estimation, crée le diagramme de Gantt.
Développeur	avoir des modèles clairs , Connaître le volume de données attendu , Avoir une API bien documentée pour les rappels (Web Push, WebSocket).	Code l’interface web (HTML/CSS/JS), intègre les notifications web (Push API). Corrige les bugs d’affichage responsive. Crée l’API CRUD, programme le moteur de rappel (côté serveur).
Testeur	Disposer de cas de test	Rédige des scénarios de test, exécute des tests manuels/automatisés, remonte les anomalies sur les délais de rappel.

3. Périmètre MoSCoW :

Fonctionnalité	Description	Priorité
ajouter une tâche +	un titre avec la date et la description	Must
supprimer une tâche —	supprimer définitivement une tache	Must
Recevoir un rappel 💡	Notification web web (toast, alert) à l'heure choisie par l'utilisateur	Must
marquer une tache ✅	cocher comme faite	Should

Modifier une tâche 📌	Changer titre, date, description	Should
Catégoriser les tâches 🗂️	une tache personnel, travail, urgent , école, etc.	could
Partager une tâche 📄	Envoyer à un autre utilisateur	could

4. Contraintes:

- **Langage** : html css javascript(frontend) php(backend) mysql (db)
- finir le projet et le déposer avant la semaine 12
- Aucune donnée sensible (mot de passe, localisation) ne doit circuler en clair.

5. dépendances externes :

- Apache HTTP Server via XAMPP pour exécuter php et héberger le site
- mysql pour avoir une base de donnée qui stock (les utilisateurs, les notifications et les taches)
- API Notification du navigateur pour afficher les notification à l'utilisateur

6. cas d'utilisation :

→ Cas 1:

Acteur : l'utilisateur

Précondition : être inscrit dans le site (avoir un compte email + code)

Déroulement : cliquer sur 'nouvelle tache' → choisir un titre → ajouter une description → ajouter une date , l'heure → validé.

Résultat : tache en base de données + rappel planifié

Erreur possible : heure/date invalide (passé) → message d'erreur 'impossible'

→ cas 2 :

Acteur : Système web (navigateur / service worker)

Précondition : Heure et date du rappel atteinte + utilisateur connecté + onglet/application web ouverte ou fermée (selon permission)

Déroulement : Le navigateur ou service worker déclenche une notification web (titre + message)

Résultat : utilisateur informé par le rappel du tache même si l'onglet n'est pas actif

Erreur possible : permissions des notifications désactivé → un message est affiché dans l'interface web (ex: bandeau, toast, zone de notification interne)

->cas3:

Acteur: l'utilisateur

Précondition : Application web ouverte + une tâche existante

Déroulement : clic long sur la tache(ou clic droit / menu contextuel) → confirmation → suppression définitive

Résultat : tache retirée de la liste + rappel annulé(coté serveur et client).

7. Architecture technique :

front	HTML , CSS , JS
back	JS(gestion des events) , php (connecter à la base de donnée) : pour rendre le site plus dynamique
BDD	MYSQL

8. les risques :

Risque	Impacte	probabilité	stratégie de mitigation
des notifications web bloqués	élevé	moyenne	Vérification des permissions Notification API, affichage d'un message dans l'interface si refusé, proposition de réactivation
bug des dates et les heures	moyen	moyenne	Utilisation de bibliothèques robustes : date-fns, day.js, ou moment.js (ou native Intl.DateTimeFormat)
perte de données	moyen	faible	Sauvegarde automatique dans localStorage + IndexedDB, appel API REST vers backend (base de données SQL/NoSQL)
délai	moyen	faible	suivi le diagramme de gantt , faire des meetings pour suivre l'avancement de chaque membre

9. Glossaire :

tache → unité de travail avec un titre , une description

rappel → notification programmé à une date et une heure par l'utilisateur

Service Worker→ Script JavaScript exécuté en arrière-plan par le navigateur, indépendant de la page web. Il permet de gérer les notifications push, les rappels et le cache même quand l'application n'est pas ouverte.

Notification web→ Message affiché par le navigateur (même si l'onglet est inactif ou fermé) grâce à la Notification API. Peut être un toast, une alerte système ou un message dans l'interface.

Sprint → une période de travail

API REST→ interface de programmation qui permet la communication entre le frontend, et le backend

Permission→ c'est une demande au navigateur pour que le site puisse envoyer des messages aux utilisateurs

LocalStorage / IndexedDB→ c'est un mécanisme de stockage coté navigateur qui permet le sauvegarde des données localement sur l'ordinateur d'utilisateur .

10. Hypothèse et exclusion :

Hypothèse	
Le projet sera développé avec une architecture frontend (HTML/CSS/JS) + backend (API REST) classique.	Le chef de projet dispose d'outils de suivi (diagramme de Gantt, tableau de bord) et les membres sont disponibles pour les réunions régulières.
Exclusions	
Pas de mode complètement hors ligne	Pas de collaboration multi-utilisateurs

11. KPIs:

- Taux de réussite des notifications : >= 95%
- Temps moyen pour créer une tâche avec un rappel : <= 10 secondes
- Temps de réponse de l'application web : <= 2 secondes
- Nombre de bugs rencontrés lors des tests : <= 5%
- Respect du planning : ≤ 1 jour de retard total

12.Planification macro :

Phase	Activité	Livrable
S1 - S2	Spécification	Mini-CDC
S3-S5	Conception	Maquettes
S6-S9	Développement	Code, base de donnée , notifications
S10-S11	Test	Test + Intégration
S12	Finalisation	Démonstration

13. Critère de succès :

- L'application web se charge dans le navigateur sans aucune erreur ou crash
- notification web (via Notification API) s'affiche à la date/heure programmée, même si l'onglet est inactif
- La liste des tâches est toujours présente après avoir fermé et rouvert le navigateur
- L'utilisateur peut modifier n'importe quel champ d'une tâche existante et peut supprimer définitivement une tâche avec confirmation
- L'utilisateur peut ouvrir, fermer et rouvrir l'application (onglet ou fenêtre) plusieurs fois sans rencontrer d'erreur ni de blocage

14. Wireframes :

écran	nom	contenu
1	inscription / connexion	un formulaire avec des champs "Email / Nom d'utilisateur + Mot de passe Bouton" "Se connecter" (si compte existe) , Lien "Créer un compte" (si compte n'existe pas)
2	principale	Liste des tâches avec : Titre, Description courte, Date/Heure d'échéance Icône ou bouton + pour ajouter Possibilité de modifier/supprimer (clic long ou menu contextuel)
3	ajout	Champ Titre (obligatoire) Champ Description (optionnel) Champ Date et Heure (sélecteur natif du navigateur) Bouton "Enregistrer"
4	notification	Notification web (toast ou notification système) selon permission

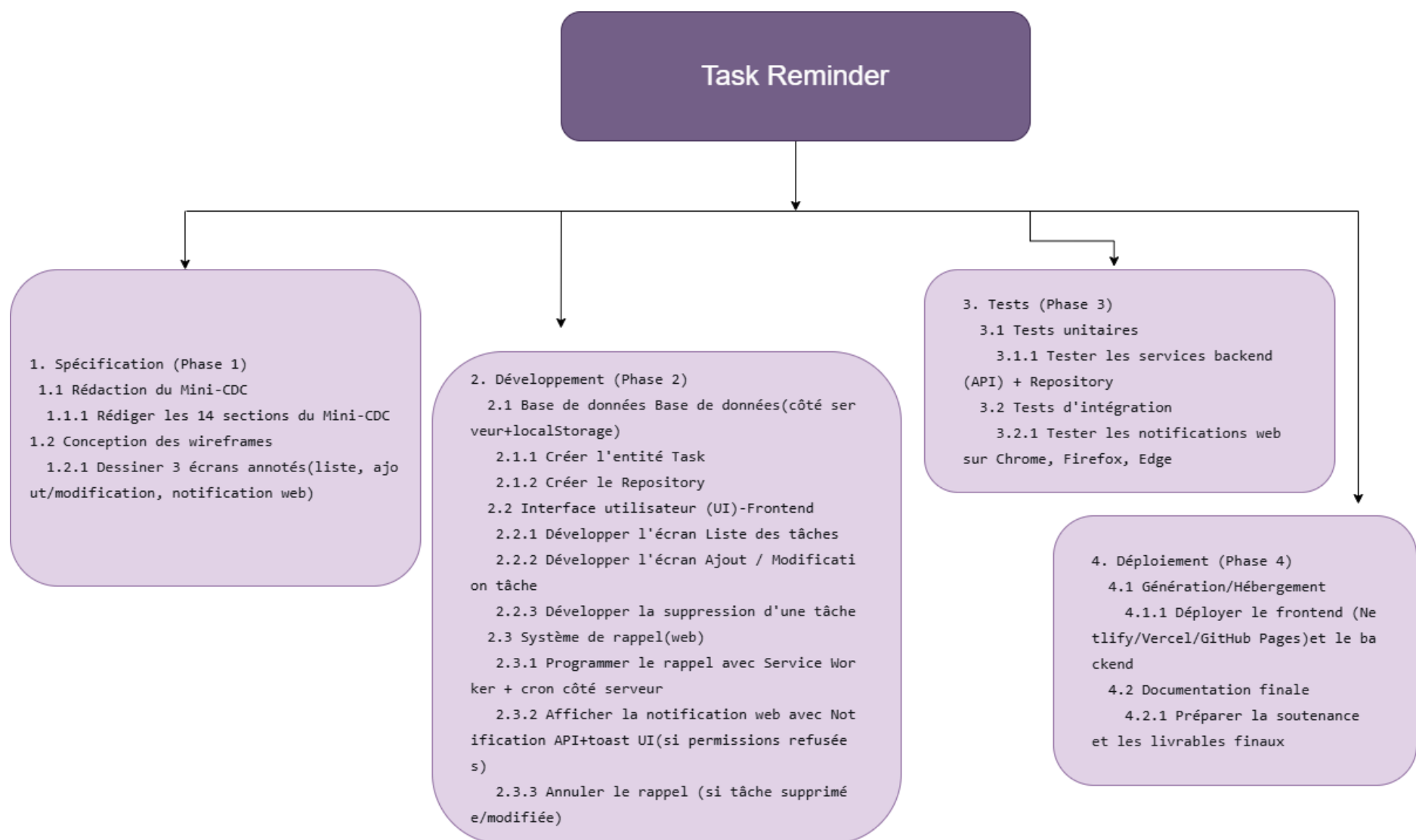
II.Work Breakdown Structure WBS :

1. Tableau WBS:

Niveau	Nom	Contenu	Durée indicative
0	Projet	Task Reminder	12 semaines
1	Phase 1	Spécification	2 semaines
2	Lots 1.1	Rédaction des besoins	1 sem
3	Tâches 1.1.1	Rédaction user stories + cas d'utilisation + validation	5jr
2	Lots 1.2	Conception technique	1sem
3	Tâches 1.2.1	UI/UX + modèle BDD + endpoints API	5 jr
1	Phase 2	Développement	7sem
2	Lots 2.1	Backend – API CRUD	2 sem
3	Tâches 2.1.1	Création BDD + API (POST/GET/PUT/DELETE)	10jr
2	Lots 2.2	Backend – Moteur de rappels	1.5sem
3	Tâches 2.2.1	Vérificateur échéances + déclenchement notifications web	5 jr
3	Tâches 2.2.2	Gestion des permissions utilisateur	3 jr

2	Lots 2.3	Frontend – Interface utilisateur	2 sem
3	Tâches 2.3.1	Écrans (login, liste, ajout/modification)	7jr
3	Tâches 2.3.2	Notifications web (toast) + responsive design	5 jr
2	Lots 2.4	Frontend – Persistance	1.5 sem
3	Tâches 2.4.1	Sauvegarde locale (LocalStorage) + synchro API	5jr
3	Tâches 2.4.2	Gestion des erreurs réseau	3jr
1	Phase 3	Tests	2 sem
2	Lots 3.1	Tests fonctionnels	1sem
3	Tâches 3.1.1	Rédaction cas de test + tests manuels (CRUD, notifications)	5jr
2	Lots 3.2	Tests non-fonctionnels	1sem
3	Tâches 3.2.1	Tests multi-navigateurs + régression + correction bugs	5jr
1	Phase 4	Déploiement	1jsem
2	Lots 4.1	Mise en production	3jr
3	Tâches 4.1.1	Configuration hébergement + déploiement	3jr
2	Lots 4.2	Livrable finale	2jr
3	Tâches 4.2.1	Validation critères de succès (recette utilisateur)	2jr

2. L'arbre WBS:



III. Estimation du projet :

Par analogie :

Projet de référence choisi:

Application web « *ToDoList avec rappels* »

Durée réelle : **12 semaines**

Calcul :

$$D(\text{projet}) = 12 * 0,75 * 1,10 * 1,15 * 1,05$$

$$D(\text{projet}) = 12 * 0,997 = 12 \text{ semaines}$$

Effort estimé :

12 semaines = 3 mois

Avec une équipe de 4 personnes :

$$\text{Effort} = 3 * 4 = 12 \text{ mois-hommes}$$

Résultat :

Méthode	Effort estimé	Durée estimée	Equipe
Analogie	12 mois-hommes	12 semaines	4 personnes

IV. Livrable 4 : Diagramme de Gantt

Le fichier du diagramme .gan sera envoyé après le Livrable .

V. Récapitulatif des livrables :

N°	Livrable	Volume	Format
1	Mini-Cahier des Charges (14 sections + wireframes)	6 pages	PDF
2	WBS (arbre 3 niveaux + dictionnaire	3 pages	PDF
3	Estimation (COCOMO et/ou Analogie - calculs détaillés)	2 page	PDF
4	Diagramme de Gantt + tableau des marges	1 page	.gan
Total séance		12 pages	PDF